ACTIVIDAD DE SISTEMAS OPERATIVOS

PHP2Shell desde dentro

Informe de laboratorio: memoria, procesos y archivos

Ejecución documentada	23 de octubre de 2026, 02:25–02:25:40 (UTC-04)
Equipo	Kali Linux · Docker · PHP 8.5.11-FPM · nginx
Objetivo	`http://127.0.0.1:18080/`
Entregable	Este único PDF

Este documento reúne el proceso completo, las capturas de la evidencia y el análisis de las tres capas que participan en la caída: memoria de usuario, procesos y sistemas de archivos.

> *“Si no está en la evidencia, es una opinión.”*

1. Resumen ejecutivo

El laboratorio reproduce un agotamiento de memoria controlado en el arranque de una petición PHP, dentro del manejo de `session.upload_progress`. La conclusión central es que se trata de un **OOM de PHP**, no de un OOM del kernel:

- PHP alcanza su límite de **4 MiB** (`memory_limit=4M`) y escribe un fatal en `Unknown on line 0`.
- El worker de PHP-FPM termina con **`exit code 70`** (`exit_group(70)`), que es un estado de salida, no una señal.
- El proceso maestro de FPM sigue vivo y crea un worker nuevo.
- nginx corta la conexión FastCGI y responde **`502 Bad Gateway`**.
- El cgroup del contenedor nunca se acerca a su límite de **256 MiB**. La prueba exacta es `memory.events` con `oom 0` y `oom_kill 0`.
- La sesión queda en `/tmp`, un `tmpfs` de 64 MiB, con modo `600` y propietario `www-data`.

Las tres capas cuentan la misma historia desde lugares distintos. Esa correlación es el corazón de la actividad.

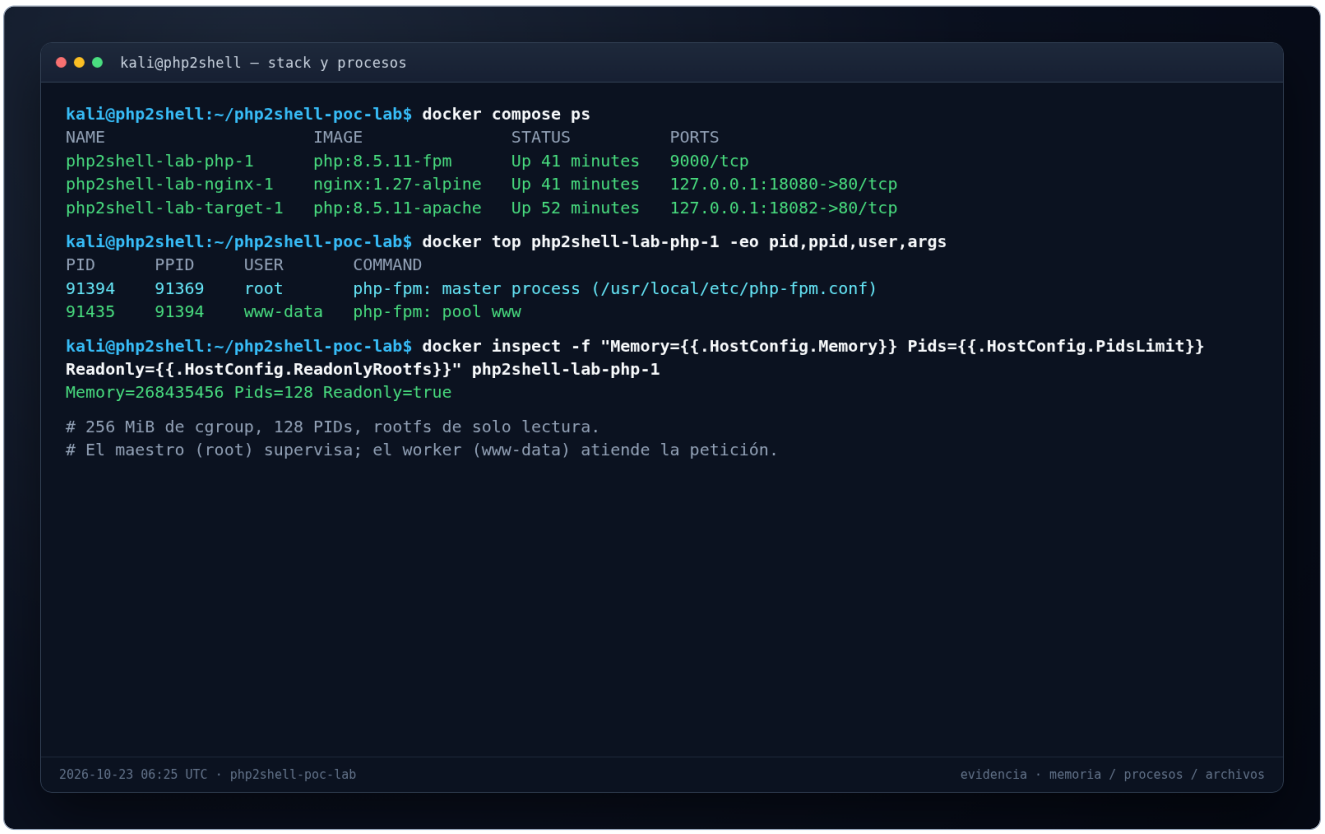
2. Entorno y configuración

La corrida documentada se hizo sobre el stack vulnerable del acto 1. Se usó el puerto **18080** para no chocar con servicios que ya estaban escuchando en el equipo.

Dato	Valor	Fuente
Sistema anfitrión	Kali Linux 7.1.5+kali-amd64	`uname -a`
Contenedor PHP	`php2shell-lab-php-1`	`docker compose ps`
Imagen PHP	`php:8.5.11-fpm`	`docker inspect`
Proxy	nginx 1.27.5, FastCGI a `php:9000`	logs de nginx
URL	`http://127.0.0.1:18080/`	`curl`
`memory_limit`	`4M` (4.194.304 bytes)	`php -i`
`session.save_path`	`/tmp`	`php -i`
`session.upload_progress`	`On`, `cleanup=Off`	`php -i`
`memory.max` del cgroup	268.435.456 bytes (256 MiB)	cgroup v2
`pids.max`	128	cgroup v2
Rootfs	solo lectura (`ReadonlyRootfs=true`)	`docker inspect`
`/tmp`	`tmpfs`, 64 MiB, `noexec`, `inode64`	`findmnt`
Pool FPM efectivo	`pm=static`, `pm.max_children=1`	`zz-php2shell-lab.conf`

> **Ojo con `php -i`:** el binario CLI no lee la configuración del pool FPM. La configuración efectiva del pool está en `/usr/local/etc/php-fpm.d/zz-php2shell-lab.conf`; por eso conviene revisar el archivo dentro del contenedor y no quedarse solo con la salida de CLI.

Captura 1 — Stack y procesos



Captura 1. `docker compose ps`, `docker top` e inspección de límites: el maestro corre como root y el worker como www-data.

La captura muestra la separación que importa: el **maestro** es el supervisor y corre como `root`; el **worker** atiende la petición como `www-data`. También confirma los límites del contenedor: 256 MiB de memoria, 128 PIDs y un rootfs de solo lectura.

3. Proceso seguido

La actividad se puede ejecutar de una vez con el orquestador, o paso a paso. En esta corrida se usó el flujo completo y luego se revisaron los artefactos por separado.

1. Se inició PHP-FPM y nginx en `127.0.0.1:18080`.
2. Se tomó un snapshot **baseline** con `/proc`, cgroup, montajes, sesiones y logs.
3. Se validó el handler de sesión con una petición pequeña; se creó un marcador y un `sess_*`.
4. Se trazó el worker con `strace` desde un contenedor auxiliar con `SYS_PTRACE`.
5. Se disparó el trigger benigno de **1 MiB** en modo `progress`, que fuerza el OOM de PHP.
6. Se tomó el snapshot **after** y se cruzaron los logs de FPM, nginx, la traza y la línea de tiempo de memoria.

```
# Flujo completo usado como base
LAB_PORT=18080 ./lab/os-activity/run-activity.sh \
  --start --fresh \
  --url http://127.0.0.1:18080/

# Equivalente manual, si se quiere ir por partes
./lab/os-activity/collect-snapshot.sh baseline
python3 lab/rt.py validate --url http://127.0.0.1:18080/ \
  --docker-inspect --compose-file docker-compose.yml
./lab/os-activity/trace-worker.sh 10
python3 trigger.py --url http://127.0.0.1:18080/ \
  --mode progress --size 1048576
./lab/os-activity/collect-snapshot.sh after
```

`--fresh` recrea el contenedor PHP para que `memory.peak` no arrastre picos de corridas anteriores. Un `docker compose restart php` no resetea el pico: reinicia el proceso dentro del mismo cgroup.

4. Capa de memoria: el aire que se acaba

4.1 Números observados

Métrica	Baseline	Después del OOM	Lectura
`memory.current`	11.857.920 B (11,31 MiB)	11.546.624 B (11,01 MiB)	Memoria del cgroup en cada snapshot
`memory.peak`	15.937.536 B (15,20 MiB)	17.813.504 B (16,99 MiB)	Pico histórico del cgroup
`memory.max`	268.435.456 B (256,00 MiB)	268.435.456 B (256,00 MiB)	Límite del cgroup
`pids.current` / `pids.max`	2 / 128	2 / 128	Solo maestro y worker
`VmSize` del worker	191.504 kB	191.504 kB	Espacio virtual reservado
`VmRSS` del worker	15.776 kB (15,41 MiB)	11.188 kB (10,93 MiB)	Páginas residentes
`RssAnon`	7.724 kB	7.716 kB	Memoria anónima
`RssFile`	7.144 kB	3.468 kB	Archivos mapeados
`memory.events`	`oom 0`, `oom_kill 0`	`oom 0`, `oom_kill 0`	Sin OOM del kernel

La muestra más alta de `memory.current` durante el muestreo de 200 ms fue **13.418.496 bytes (12,80 MiB)**. El pico histórico subió a **16,99 MiB**. Nada de eso se acerca a los 256 MiB del cgroup.

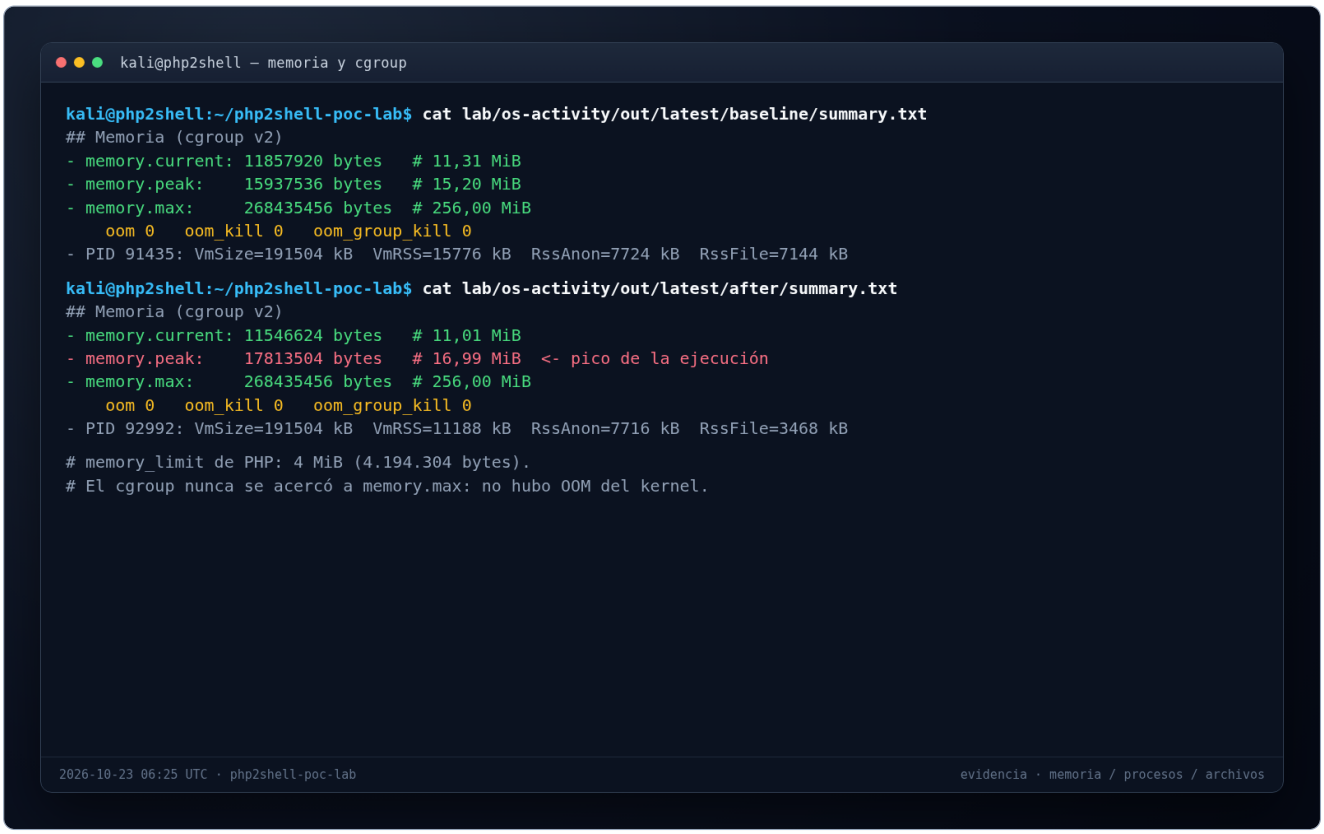
4.2 Por qué `memory_limit` no es la memoria del proceso

`memory_limit` es un contador del **Zend Memory Manager**: cuenta las asignaciones que maneja PHP. No cuenta todo lo que el proceso tiene mapeado: bibliotecas compartidas, estructuras del propio FPM, pilas, mappings del sistema y páginas que el kernel conserva. Por eso el worker puede tener `VmRSS` de 11–16 MiB y, al mismo tiempo, `memory_limit=4M`.

Esa diferencia no es un error de medición. Es la razón por la que hay que mirar varias métricas a la vez:

- **`memory_limit`**: cuándo PHP decide abortar la petición.
- **`VmRSS`**: cuánta memoria física está usando el proceso.
- **`memory.current`**: cuánta memoria carga el kernel al cgroup.
- **`memory.max`**: cuándo el kernel tendría la última palabra.

Captura 2 — Baseline, después y eventos del cgroup



Captura 2. Memoria del cgroup y del worker antes y después del OOM; `memory.events` sigue en cero.

La captura deja ver la aparente contradicción: PHP dice que se quedó sin 4 MiB, pero el cgroup tiene 256 MiB y ni se inmuta. La explicación está en la tabla anterior: son contadores distintos y miden cosas distintas.

5. Capa de traza: la secuencia exacta

La traza se tomó con un contenedor auxiliar que comparte el espacio de PIDs del contenedor PHP y tiene `CAP_SYS_PTRACE`. El equipo anfitrión no tenía `strace` instalado, así que esa fue la forma limpia de observar al worker sin instalar nada en el host.

Líneas clave de la traza:

```
7  06:25:36.781971 read(5<TCPv6:[342562]>, "------php2shellLabBoundary ...
name=\"PHPSESSID\" ...", 5120) = 5120
7  06:25:37.827806 mmap(NULL, 2097152, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fe14c000000
7  06:25:37.830076 write(2<pipe:[340030]>, "NOTICE: PHP message: PHP Fatal error:  Allowed memory
size of 4194304 bytes exhausted (tried to allocate 1052672 bytes) in Unknown on line 0\nStack
trace:\n#0 {main}\n", 164) = 164
7  06:25:37.849131 exit_group(70)    = ?
```

La secuencia se lee de izquierda a derecha:

1. El worker lee el cuerpo multipart de la petición.
2. Zend MM pide un bloque de **2 MiB** con `mmap`.
3. El siguiente intento de **1.052.672 bytes** ya no cabe en el límite de 4 MiB.
4. PHP escribe el fatal y termina con `exit_group(70)`.

`exit_group` es una llamada al sistema que termina todos los hilos del proceso. El `70` es el estado de salida; no es una señal. Un `SIGKILL` se vería distinto en la traza y en los logs, y además dejaría rastro en `memory.events` si lo hubiera enviado el OOM killer del kernel.

Captura 3 — Traza del OOM

kali@php2shell — traza del worker

```
kali@php2shell:~/php2shell-poc-lab$ grep -E "read\(5|mmap\(NULL, 2097152|Fatal error|exit_group" lab/os-activity/out/latest/trace/strace-*.log
7  06:25:36.781971 read(5<TCPv6:[342562]>, "------php2shellLabBoundary ... name="PHPSESSID" ...", 5120) = 5120
7  06:25:36.786284 read(5<TCPv6:[342562]>, "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...", 80) = 80
7  06:25:37.827806 mmap(NULL, 2097152, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe14c000000 <0.001108>
7  06:25:37.830076 write(2<pipe:[340030]>, "NOTICE: PHP message: PHP Fatal error:  Allowed memory size of 4194304 bytes exhausted (tried to allocate 1052672 bytes) in Unknown on line 0\nStack trace:\n#0 {main}\n", 164) = 164
7  06:25:37.849131 exit_group(70)    = ?

# El worker leyó el multipart, Zend MM pidió un bloque de 2 MiB,
# el siguiente intento de 1.052.672 bytes ya no cupo y terminó con exit_group(70).
# No hay SIGKILL ni intervención del OOM killer en esta traza.
```

2026-10-23 06:25 UTC · php2shell-poc-lab

evidencia · memoria / procesos / archivos

Captura 3. La traza muestra la lectura del multipart, el `mmap` de 2 MiB, el fatal de PHP y `exit_group(70)`.

6. Capa de procesos: quién muere y quién lo reemplaza

6.1 Árbol antes y después

Dato	Antes	Después
Maestro FPM (PID del host)	91394	91394
Worker FPM (PID del host)	91435	92992
Worker FPM (PID dentro del contenedor)	7	132
PPID del worker	91394	91394
Usuario del maestro	`root`	`root`
Usuario del worker	`www-data`	`www-data`
Estado del worker	`S` (sleeping)	`S` (sleeping)
Hilos del worker	1	1

El maestro no cambia: es el proceso que supervisa el pool. El worker sí cambia porque el fatal lo termina y FPM crea uno nuevo para seguir atendiendo.

6.2 Logs que cuentan la historia

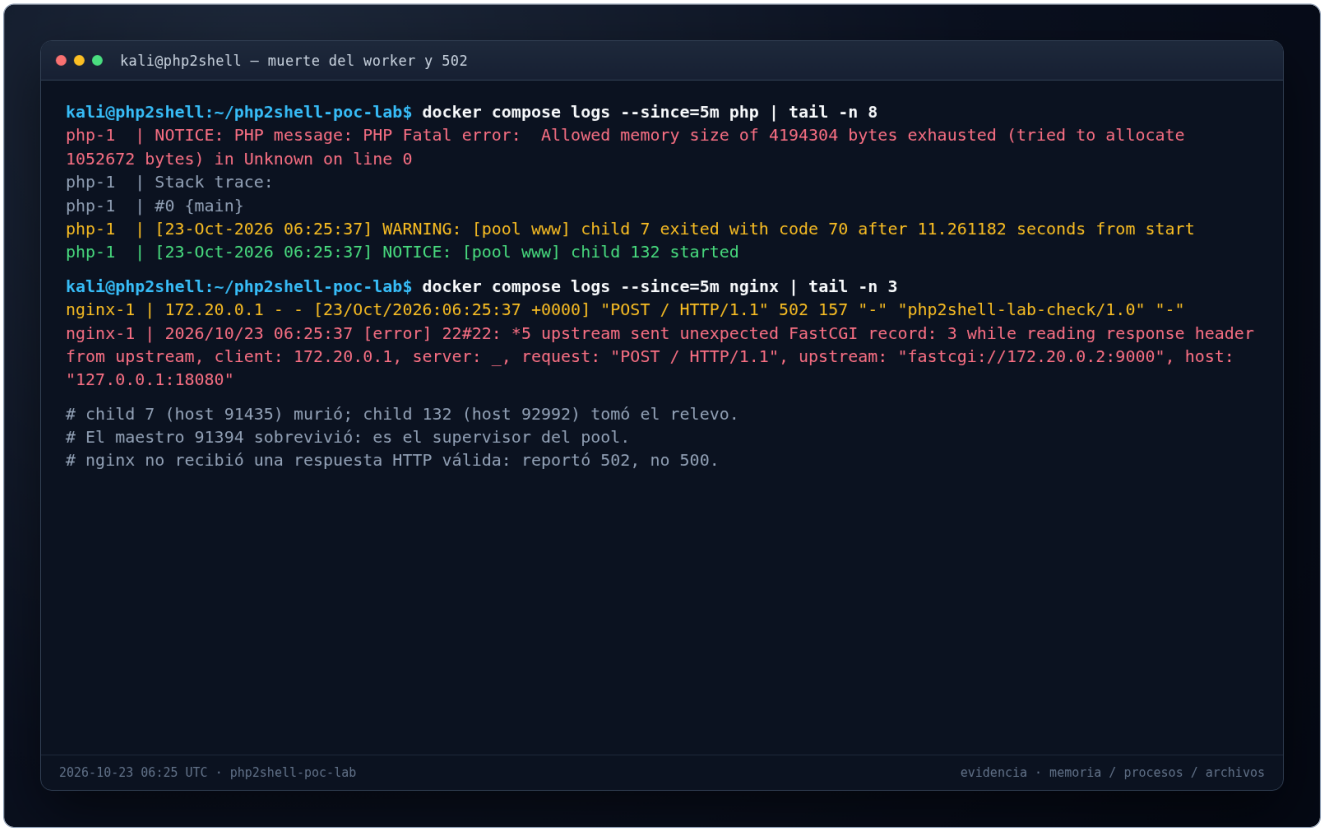
```
php-1  | NOTICE: PHP message: PHP Fatal error:  Allowed memory size of 4194304 bytes exhausted
(tried to allocate 1052672 bytes) in Unknown on line 0
php-1  | [23-Oct-2026 06:25:37] WARNING: [pool www] child 7 exited with code 70 after 11.261182
seconds from start
php-1  | [23-Oct-2026 06:25:37] NOTICE: [pool www] child 132 started

nginx-1 | 172.20.0.1 - - [23/Oct/2026:06:25:37 +0000] "POST / HTTP/1.1" 502 157 "-" "php2shell-
lab-check/1.0" "-"
nginx-1 | 2026/10/23 06:25:37 [error] 22#22: *5 upstream sent unexpected FastCGI record: 3 while
reading response header from upstream
```

El código **70** corresponde a `EX_SOFTWARE` en la convención de `sysexits.h`. FPM lo registra como salida del hijo, no como señal. El maestro reacciona creando el hijo 132.

nginx devuelve **502**, no 500. Un 500 sería una respuesta de error generada por la aplicación; un 502 indica que el upstream FastCGI no entregó una respuesta válida. En este caso, el worker murió a mitad del intercambio.

Captura 4 — Muerte, reemplazo y 502



Captura 4. El log de FPM muestra el fatal, `child 7 exited with code 70` y `child 132 started`; nginx registra el 502.

7. Capa de archivos: lo que queda escrito

7.1 Montajes

Ruta	Tipo	Opciones relevantes	Tamaño	Escritura
`/tmp`	`tmpfs`	`rw,nosuid,nodev,noexec,relatime,size=65536k,inode64`	64 MiB	sí
`/var/www/html`	`ext4` (bind mount)	`ro,relatime,errors=remount-ro`	rootfs	no
`/var/www/html/uploads` (target)	`tmpfs`	`rw,nosuid,nodev,noexec`	16 MiB	sí

El rootfs de solo lectura se comprueba con un `touch`, que falla con `Read-only file system`. `/tmp` sí es escribible porque es un `tmpfs` separado.

7.2 La sesión

```
inodo=3  modo=600  propietario=www-data:www-data  tamaño=362
ruta=/tmp/sess_97cc4566e2540eb35ca39409c24fafba
```

El archivo sobrevive a la muerte del worker porque no vive en la memoria del proceso: vive en el `tmpfs` del contenedor. El modo `600` permite lectura y escritura solo al propietario (`www-data`).

7.3 La prueba del inodo

```
primer_touch  inodo=5
segundo_touch inodo=6
```

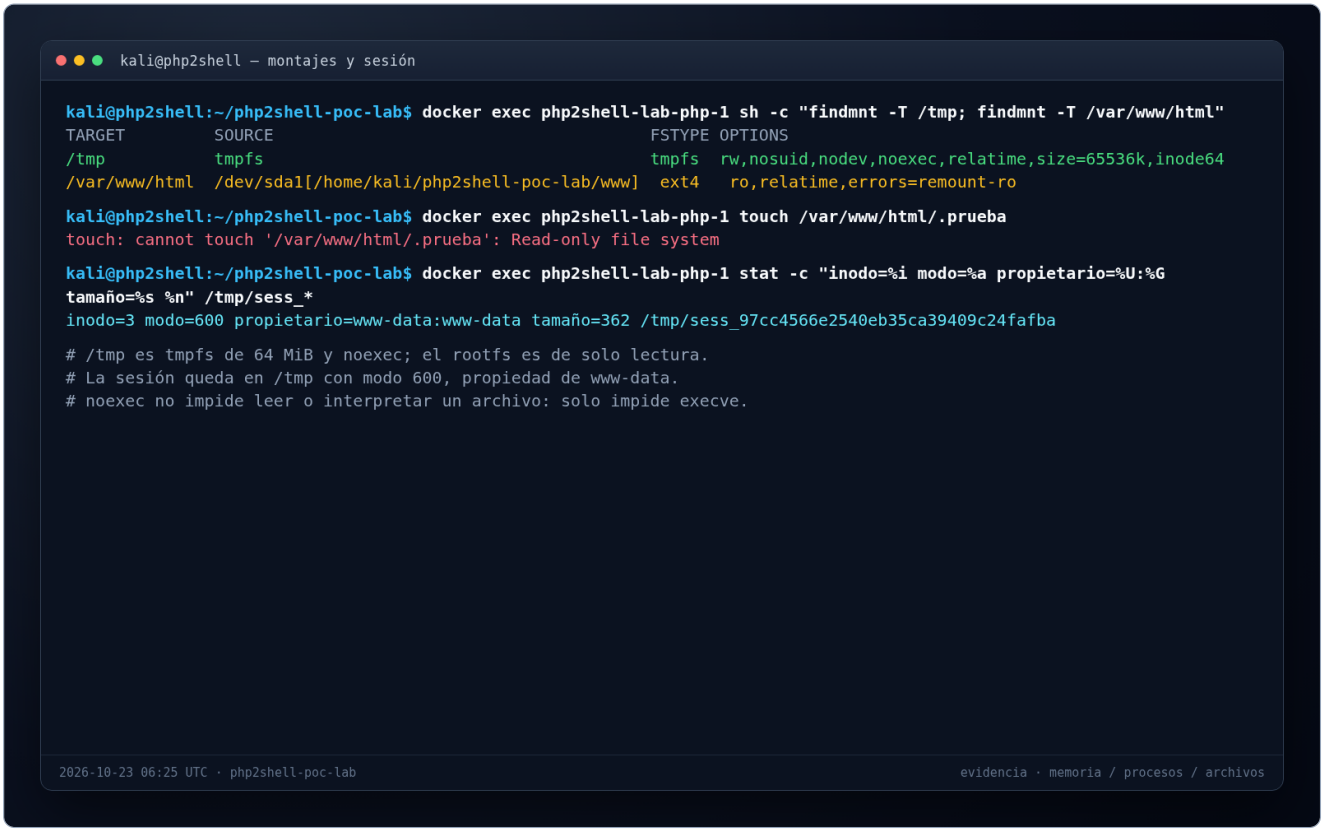
En esta corrida el inodo no se reutilizó. Eso no es una regla universal: el número depende del asignador del `tmpfs` y del estado del sistema. Lo importante es entender que un inodo identifica un objeto del sistema de archivos, mientras que el nombre es una entrada de directorio. Borrar un nombre no significa borrar el objeto si otro proceso lo mantiene abierto.

7.4 La trampa de `noexec`

`noexec` impide ejecutar binarios o scripts con `execve` desde ese montaje. No impide escribir, leer, incluir o interpretar un archivo. Un archivo PHP en un directorio `noexec` puede ser leído e interpretado por Apache/mod_php; el `execve` que falla o funciona es el del servidor web, no el del archivo.

Esta actividad **no** incluye la cadena de subida a webshell ni RCE. La comparación con el target de entrenamiento sirve solo para separar dos ideas: “escribir un archivo” y “ejecutar un archivo”.

Captura 5 — Montajes, rootfs y sesión



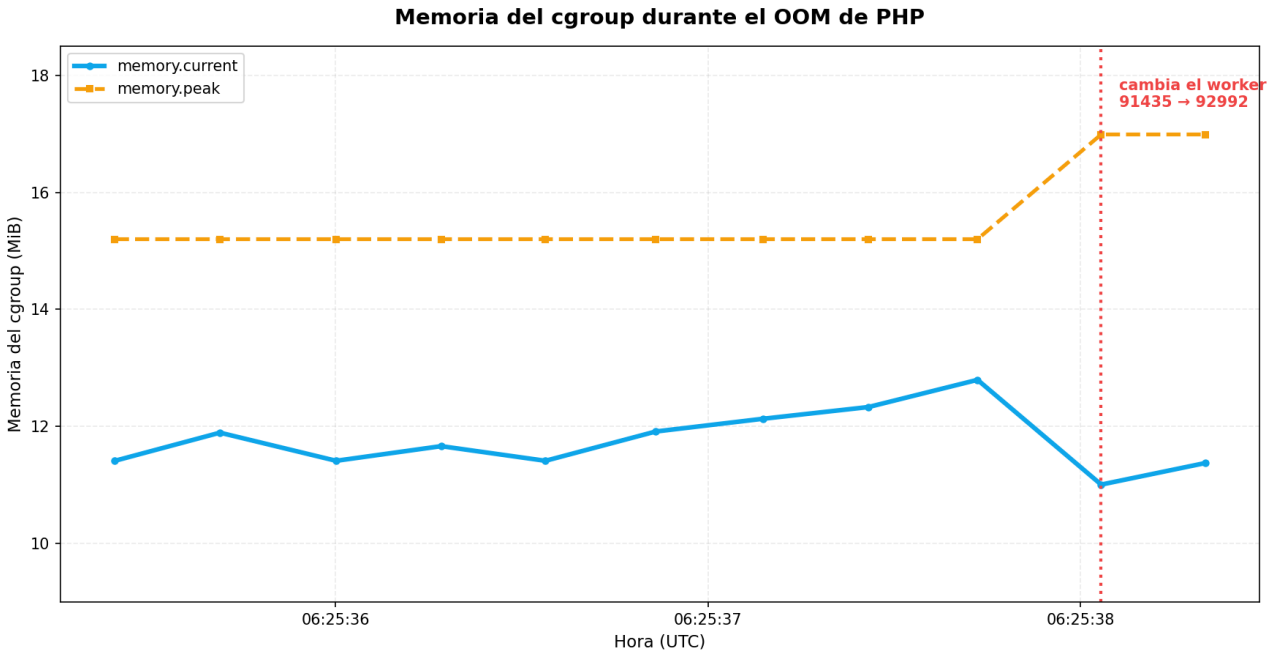
Captura 5. `/tmp` es `tmpfs` con `noexec`; `/var/www/html` es de solo lectura; la sesión queda en `/tmp` con modo 600.

8. Línea de tiempo: una caída, tres relojes

Hora (UTC)	Evento	Capa	Evidencia
06:25:35.408	Primer sample del baseline	memoria	`11,41 MiB` en `memory.current`
06:25:36.782	El worker lee el multipart	red / proceso	`read(5...) = 5120`
06:25:37.827	Zend MM pide 2 MiB	memoria de usuario	`mmap(NULL, 2097152, ...)`
06:25:37.830	Fatal por `memory_limit`	memoria de usuario	`Fatal error ... 4194304 bytes exhausted`
06:25:37.849	El worker termina	proceso	`exit_group(70)`
06:25:37	FPM registra y reemplaza	proceso	`child 7 exited with code 70` / `child 132 started`
06:25:37	nginx ve el corte	red / proceso	`502` / `unexpected FastCGI record`
06:25:39	Snapshot final	filesystem	`sess_*` en `/tmp`

La línea de tiempo no es una suma de eventos aislados: es la misma caída vista desde tres relojes. El fatal de PHP, la salida del worker y el 502 de nginx ocurren en el mismo segundo. La sesión queda escrita un poco después, cuando ya no hay worker que atienda la petición original.

Captura 6 — Memoria durante la caída



Captura 6. `memory.current` sube hasta 12,80 MiB, el pico llega a 16,99 MiB y el worker cambia de 91435 a 92992.

La gráfica está ampliada para ver el movimiento real. El límite del cgroup es 256 MiB, así que la subida es pequeña en términos del contenedor. El pico histórico queda registrado aunque el worker haya muerto; eso explica por qué `memory.peak` no baja después del OOM.

9. Conclusiones

Memoria

1. El fatal lo produce **PHP**, no el kernel: `memory_limit=4M` se agota y el mensaje aparece en `Unknown on line 0`.
2. `memory.current` y `memory.peak` se mantienen muy por debajo de `memory.max`; la métrica que lo confirma es `memory.events` con `oom 0` y `oom_kill 0`.
3. `VmSize`, `VmRSS` y `memory_limit` no son intercambiables: miden espacio virtual, memoria residente y asignaciones gestionadas por Zend, respectivamente.

Procesos

1. El que muere es el **worker**, no el maestro. El maestro sobrevive porque es el supervisor del pool.
2. `exit code 70` es un estado de salida (`exit_group(70)`), no una señal como `SIGKILL`.
3. nginx responde **502** porque el upstream FastCGI se rompió; no es un 500 generado por la aplicación.

Sistemas de archivos

1. La sesión se escribe en `/tmp`, un `tmpfs` de 64 MiB, y sobrevive a la muerte del worker.
2. El archivo tiene modo `600` y propietario `www-data`; el rootfs de solo lectura no se toca.
3. `noexec` no impide leer o interpretar un archivo. La actividad no explota esa diferencia: solo la documenta.

La evidencia que cierra el caso

```
memory.current : 11.546.624 B  (11,01 MiB)
memory.peak    : 17.813.504 B  (16,99 MiB)
memory.max     : 268.435.456 B (256,00 MiB)
memory.events  : oom 0, oom_kill 0
```

Con esos números no se puede sostener que el kernel haya matado el proceso. La caída es de memoria de usuario y el resto de las capas reacciona a esa caída.

10. Reproducción

```
cd php2shell-poc-lab

# 1) Iniciar el stack (si el 8080 está ocupado, usar 18080)
docker compose up -d php nginx

# 2) Corrida completa: baseline, sesión, traza, OOM y snapshot final
LAB_PORT=18080 ./lab/os-activity/run-activity.sh \
  --start --fresh \
  --url http://127.0.0.1:18080/

# 3) Revisar evidencia
ls -la lab/os-activity/out/latest/
cat lab/os-activity/out/latest/summary.md

# 4) Detener y limpiar
docker compose down -v
```

Si no se quiere usar `strace`, el flujo también funciona con `--no-trace`; en ese caso la evidencia de memoria y procesos se obtiene de `/proc`, cgroup, `pidstat` y logs.

11. Anexo: evidencia y entregable

La corrida documentada dejó su evidencia en:

```
lab/os-activity/out/20261023T062527Z-activity/
├── baseline/
├── after/
├── trace/
│   ├── strace-20261023T062535Z-pid7.log
│   ├── trace-meta.txt
│   └── summary.txt
├── memory-timeline.tsv
├── validate.log
├── trace-console.log
└── summary.md
```

Este PDF es el **único entregable**. El archivo `worksheet.md` queda como borrador de trabajo y no se entrega. Las capturas incluidas son imágenes de la evidencia recolectada; si se repite el laboratorio, los PIDs, los tiempos y los inodos van a cambiar, pero la relación entre las tres capas se mantiene.

Lista de verificación

- ☐ Entorno y límites documentados.
- ☐ Baseline y snapshot final.
- ☐ Traza con `mmap`, fatal y `exit_group(70)`.
- ☐ Logs de FPM con salida y reemplazo del worker.
- ☐ Log de nginx con 502.
- ☐ Montajes y archivo de sesión.
- ☐ Línea de tiempo y análisis de las tres capas.
- ☐ Comandos reproducibles.
- ☐ PDF único como entregable.